Model, Documentation, and Visualization Fixes

Companion to the Face Image Quality Prediction Technical Report

Face Quality Prediction Project — Fix Log

July 9, 2026

Abstract

An audit of the trained quality-prediction model, its evaluation artifacts, and its documentation against the ground-truth OFIQ data surfaced ten items: three model deficiencies (regression to the mean at the score extremes, unused component labels, training below native image resolution), one training-stability issue (periodic validation-error spikes), four documentation contradictions, and two visualization gaps. This document states each problem, the evidence for it, and the fix applied. Code fixes land in `train_quality.py`, `evaluate.py`, and the report's figure scripts; documentation fixes land in `REPORT.md`. Fixes 1–2 and 9–10 change training behaviour and take full effect on the next training run; all others are already in effect.

Note on dropout. The architecture does contain dropout — spatial dropout (`Dropout2d`, $p = 0.10$) inside every residual block and two head dropout layers ($p = 0.30$) — but dropout is active only during the training (weight-update) passes. In eval mode (`model.eval()`) PyTorch disables all dropout layers automatically, so validation, logged metrics, `evaluate.py`, and deployment inference all run dropout-free. Wherever this document or the technical report says "dropout on/off", it refers to this measurement/inference distinction, never to an architecture change: every run trains with dropout and evaluates without it.

1 Regression to the Mean at Score Extremes

Problem. The validation scatter (`eval_scatter_full.png`) shows predictions pulled toward the mean of the label distribution: images with true score below $\sim$16 are systematically over-predicted, and images above $\sim$29 are under-predicted. Three causes compound: (i) only 0.37% of labels fall below 15 and 1.0% above 30, so the mean-squared-error loss is dominated by the mid-range bulk; (ii) MSE itself is mean-seeking under label scarcity; and (iii) the sigmoid output head saturates exactly where the rare labels live. For the assumed downstream use — a quality critic inside a GAN loss — this is the worst possible failure mode, because the critic's gradient is weakest precisely on the worst images.

Fix. Three complementary changes:

1. Tail-weighted loss (`train_quality.py`, new `--tail-weight` flag). Per-sample weights $w(y) \propto \mathrm{freq}(y)^{-p}$ are computed from the train-split label histogram (20 bins), normalised so their expectation over the data distribution is 1, and capped at $5\times$. With $p = 0.5$ (inverse-square-root), rare extreme scores contribute proportionally more gradient without destabilising the loss scale. Default $p = 0$ preserves the existing behaviour exactly.
2. Linear output head (new `--head linear` flag). Replaces the final sigmoid with identity, removing saturation at the range ends. The head type is stored in the checkpoint; `evaluate.py` clamps predictions to the valid scaled range at inference, so the contract is unchanged.
3. Post-hoc isotonic calibration (`evaluate.py`, new `--calibrate` flag). A monotone map from prediction to target is fitted with the pool-adjacent-violators algorithm (pure NumPy, no scikit-learn dependency, so it runs on the offline compute node) on one half of the validation set and scored on the other half — calibration is never fitted and evaluated on the same images. Because the correction is monotone, Spearman ρ is preserved by construction. On synthetic data with compression slope 0.60, the correction recovered a slope of 0.96 and halved the MAE. This fix requires no retraining.

Items 1–2 require a retraining run on the cluster; item 3 works with the existing checkpoint immediately.

2 Periodic Training Spikes

Problem. The training curves show sharp validation-MAE spikes at epochs 15, 20, 22, 26, and 34 — including after the learning rate had already been halved, which rules out a simple step-size explanation. The likely cause is the Pearson term of the combined loss: it is computed per batch (64 images), and when a batch happens to carry near-constant targets, the correlation denominator $\|\mathbf{p}\|\,\|\mathbf{t}\|$ approaches zero. The previous guard added a tiny $\varepsilon = 10^{-8}$ to the denominator, which keeps the value finite but still lets the gradient blow up.

Fix. The denominator is now clamped from below at 10^{-4} rather than ε-padded:

$$r = \frac{\sum_i p_i t_i}{\max\big(\|\mathbf{p}\|\,\|\mathbf{t}\|,\, 10^{-4}\big)}.$$

On normal batches the denominator is orders of magnitude above the clamp, so behaviour is unchanged; on degenerate low-variance batches the gradient magnitude is now bounded.

3 False "No Early-Stopping Leakage" Claim

Problem. `REPORT.md` stated the 7,000 validation images were "never seen during training, not even for early stopping decisions." The code contradicts this: validation MSE drives both early stopping and the `ReduceLROnPlateau` scheduler every epoch. Using validation data as a model-selection signal is standard, defensible practice — but the claim as written was factually wrong and would not survive review.

Fix. Rewritten to the accurate formulation: the validation images were never used for gradient updates; they do drive early stopping and learning-rate scheduling, which leaks no pixels or labels into the weights.

4 Spearman ρ Misinterpretation

Problem. `REPORT.md` described $\rho = 0.867$ as "the model ranks images in the correct quality order 86.7% of the time." Spearman ρ is a rank correlation coefficient on $[-1, 1]$, not a percentage of correctly ordered pairs (the statistic resembling that reading is Kendall's τ, and even τ is not literally a percentage).

Fix. Reworded: $\rho = 0.867$ indicates strong monotonic agreement between the model's quality ranking and OFIQ's (1.0 = identical ordering; 0 = no relationship).

5 Parameter-Count Inconsistency

Problem. `REPORT.md` (three places) and a comment in `train_quality.py` claimed the model has ~ 0.5 M parameters; the training log prints the actual count, 0.33 M.

Fix. All four occurrences corrected to ~ 0.33 M.

6 Early-Stopping Metric: Documentation vs. Code

Problem. The `train_quality.py` docstring, the `--patience` help text, and `REPORT.md` §4.8 all said early stopping monitors validation MAE. The implementation monitors validation MSE (deliberately — it is the smoother of the two metrics epoch-to-epoch).

Fix. Docstring, help text, and `REPORT.md` corrected to validation MSE, with the rationale stated.

7 Metrics Figure Mixes Incompatible Units

Problem. `fig_metrics_summary.png` placed MAE and RMSE (native score units, values near 1.3–2.7) on the same axis as Pearson r, Spearman ρ, and R^2 (unitless, bounded by 1). The correlation metrics — the strongest results — looked visually insignificant next to the baseline-MAE bar.

Fix. `scripts/generate_figures.py` now renders two panels: error metrics in native units with the always-guess-the-mean baseline line, and correlation metrics on a dedicated $[0, 1]$ axis with the perfect-score reference at 1.0. The figure was regenerated and the technical report PDF recompiled.

8 Missing Diagnostic Visualizations

Problem. No existing figure exposes the tail bias directly, quantifies the calibration slope, or identifies which OFIQ component measures correlate with the model's errors. All three require per-image predictions, which `evaluate.py` computed but never saved.

Fix. Two changes:

1. `evaluate.py` now writes `<model>_val_preds.csv` (Filename, true, pred) by default, plus `<model>_train_preds.csv` for the train-scatter sample.
2. New `scripts/generate_diagnostic_figures.py` builds three figures from that CSV:
   - `fig_binned_residuals.png` — mean residual ± std per true-score bin; tail compression appears as positive bias at low scores and negative bias at high scores;
   - `fig_calibration.png` — least-squares slope of predicted vs. true against the ideal $y = x$;
   - `fig_component_correlation.png` — correlation of the residual with every OFIQ `.native` component measure.

   If the predictions CSV does not exist yet, the script prints the exact `evaluate.py` command to produce it and exits cleanly.

The figures themselves require one `evaluate.py` run on a machine holding the images (the cluster); everything else is in place.

9 Single-Task Training Wastes the Component Labels

Problem. The OFIQ CSV contains 27 component `.native` measures per image — occlusion, eye visibility, face margins, compression artifacts, luminance statistics, and more — but the model only ever saw the single unified score. It learns that an image is low quality without being pushed to learn why, which yields weaker features, particularly for the rare defect types that populate the tails of the score distribution.

Fix. Multi-task learning (`train_quality.py`, new `--aux K` and `--aux-weight W` flags; the job script uses `--aux 10 --aux-weight 0.3`):

1. At startup, the K component measures most correlated with the unified score are selected on the train split only. On the real data the ranking is led by `FaceOcclusionPrevention` ($|r| = 0.39$), `EyesVisible` (0.27), `MarginBelowOfTheFaceImage` (0.25), and `HeadSize` (0.20). Each is min–max scaled to $[0,1]$ with train-split ranges.
2. Both architectures gain an auxiliary head off the same pooled features, and the training loss becomes $\mathscr{L} = \mathscr{L}_{\text{main}} + W \cdot \text{MSE}(\hat{\mathbf{c}}, \mathbf{c})$ where $\mathbf{c}$ is the vector of scaled component scores.
3. The auxiliary head is a training-time regulariser only: the validation and train-eval loaders remain main-task-only, so the logged curves, early stopping, and learning-rate scheduling stay directly comparable to previous runs. `evaluate.py` and `model_architecture.py` recognise multi-task checkpoints via `ckpt["aux_cols"]` and use the unified-score output for inference.

10 Training Below Native Image Resolution

Problem. FFHQ images are native 256×256, but training resized them to 224×224 — discarding exactly the fine detail that sharpness- and compression-type quality cues depend on, before the model ever saw them.

Fix. Train at native resolution: `--img-size 256` added to `JOB_train_quality_full.sh` (the flag already existed, and adaptive pooling makes the architecture resolution-agnostic). Costs roughly 30% more GPU time per epoch. `evaluate.py` reads `img_size` from the checkpoint, so evaluation follows automatically.

11 Status Summary

Recommended next run (cluster): submit `JOB_train_quality_full.sh` — fixes 1, 9, and 10 are baked into its flags (`--tail-weight 0.5 --aux 10 --aux-weight 0.3 --img-size 256`) — then

```
python evaluate.py --model best_model_full.pt --plot eval_scatter_full.png
    --calibrate
```

#	Problem	Type	Status
1	Tail compression / regression to mean	Model	Implemented; needs retrain (`--tail-weight 0.5`, optionally `--head linear`); `--calibrate` usable now
2	Periodic training spikes	Stability	Fixed (Pearson denominator clamp)
3	Early-stopping leakage claim	Docs	Fixed in `REPORT.md`
4	Spearman ρ misread	Docs	Fixed in `REPORT.md`
5	Parameter count 0.5M vs 0.33M	Docs	Fixed in `REPORT.md` + code comment
6	Early-stop MAE vs MSE wording	Docs/code	Fixed in docstring, help, `REPORT.md`
7	Mixed-unit metrics figure	Viz	Fixed; figure regenerated, PDF recompiled
8	Missing diagnostics	Viz	Scripts ready; figures generate once the predictions CSV exists
9	Component labels unused	Model	Implemented; in job script (`--aux 10 --aux-weight 0.3`); takes effect on retrain
10	Trained below native resolution	Model	In job script (`--img-size 256`); takes effect on retrain

Compare the new binned-residual and calibration figures against the current checkpoint's to confirm the tail bias has narrowed without hurting mid-range MAE.